America's Next Top Threat Model

The Course Project

Security & Privacy in Computing

Fall 2026

Contents

1 Overview **2**
- 1.1 Project Objectives 2
- 1.2 Project Timeline 3

2 Background **4**
- 2.1 OWASP Top 10 4
- 2.2 Common Weakness Enumeration 4
- 2.3 CWE Mapping 5

3 Project Phases **5**
- 3.1 Phase I: The Engineer 5
  - 3.1.1 Software Package I: The Implementation 5
  - 3.1.2 Implementation Demo Day 6
  - 3.1.3 Phase I Deliverables and Key Dates 7
- 3.2 Phase II: The Adversary 7
  - 3.2.1 Software Package II: The Backdoors 8
  - 3.2.2 Backdoor Demo Day 8
  - 3.2.3 Phase II Deliverables and Key Dates 9
- 3.3 Phase III: The Analyst 9
  - 3.3.1 Final Presentation 9
  - 3.3.2 Phase III Deliverables and Key Dates 10

4 Check-Ins **10**
- 4.1 Check-in 1: Getting started 10
- 4.2 Check-in 2: Implementation design 11
- 4.3 Check-in 3: Backdooring a system 12
- 4.4 Check-in 4: Analyzing a system 13
- 4.5 Check-in 5: Presenting your results 13

5 System Specifications **13**
- 5.1 Automated Teller Machine (ATM) 13
- 5.2 Slot Machine 15
- 5.3 E-Voting System 16
- 5.4 Smart Thermostat 17
- 5.5 Automated Insulin Delivery (AID) System 18

Last updated: October 21, 2026

1 Overview

The cumulative final project of this course is a hands-on experience that will take you through the full lifecycle of engineering a secure system, from initial design to final security analysis. Within your teams, you will take on three primary roles, each requiring distinct strategies and a strong grasp of the concepts covered throughout the course. The roles are as follows:

- **The Engineer (Phase I):** You will design and build a system that is secure-by-design, follows industry-standard guidelines (including the OWASP Top 10 and the Common Weakness Enumeration), and meets specific functional requirements. Your team will design and implement the system, document the security features you incorporated, and be able to thoroughly explain how they satisfy the above security standards and design choices. Deliverables include *Software Package I*, which must contain your approved zipped source code and a detailed user manual. You will also give a live demonstration on Implementation Demo Day, showcasing your system's functionality and answering questions about your system to the course instructor. To guide your progress, your team will complete and submit *Check-in 1* and *Check-in 2*, which will help you navigate key questions and considerations essential to successfully keeping pace and fulfilling the project.
- **The Adversary (Phase II):** Your team will receive another group's *Software Package I*. Your goal is to identify potential weaknesses in the system and attempt to subvert and exploit them by adding backdoors to the code while maintaining the system's functionality. Your team should be able to identify which CWE and OWASP Top 10 categories the discovered vulnerabilities fall under, explain how your attacks take advantage of those weaknesses, and map to the backdoor requirements. Your Phase II deliverables are *Software Package II*, which consists of your zipped compromised system package. You will give a live presentation of this package on *Backdoor Demo Day*, during which you will demonstrate all the implemented vulnerabilities to your course instructor and answer questions regarding your design choices. Additionally, your team will complete and submit Check-in 3 to guide your progress.
- **The Analyst (Phase III):** Your team will receive another group's Software Package II and user manual. Your goal is to conduct a comprehensive security analysis of the system and identify *all* vulnerabilities introduced by the previous group. Your deliverables for this phase will include *Check-in 4* and *Check-in 5* to guide your progress, and a final group presentation. Your presentation should detail your exploration of the system, the vulnerabilities you discovered, their respective OWASP and CWE mappings, and any final conclusions. Your presentation should be delivered using a slide deck. Each team member will contribute, present, and speak for an equal portion of time. This presentation will be given to the entire class as your final submission for this project.

1.1 Project Objectives

By the end of the course project, students will:

- Demonstrate mastery of adversarial thinking by designing and applying both offensive and defensive security strategies
- Apply understanding of the OWASP Top 10, security-by-design, and the Common Weakness Enumeration (CWE) through the development, analysis, and exploitation of a system
- Integrate current application security practices into the design and development of a secure system
- Assess the security of a compromised system using industry-standard tools and methodologies

1.2 Project Timeline

Task	Date	Notes
Check-in 1	Monday, October 19	
Check-in 2	Friday, October 30	
Phase I Submission	Tuesday, November 17	
Check-in 3	Tuesday, November 17	
Implementation Demo Day	Wednesday, November 18	
Check-in 4	Sunday, November 22	
Phase II Submission	Sunday, December 6	
Backdoor Demo Day	Monday, December 7	
Check-in 5	Wednesday, December 9	
Phase III Submission	Monday, December 21	
Final Presentation	Wednesday, December 23	

2 Background

2.1 OWASP Top 10

The Open Web Application Security Project (OWASP) is an internationally recognized nonprofit organization dedicated to promoting secure software development and maintenance. The Top 10 is a list compiled from thousands of reported security incidents, designed to raise awareness about the ten most critical observed vulnerabilities and establish a strong standard of security for web-based applications. It is referenced by professionals in both academia and industry and is considered a trusted and widely adopted resource. The 2021 Top 10 is as follows:

1. Broken Access Control
2. Cryptographic Failures
3. Injection
4. Insecure Design
5. Security Misconfiguration
6. Vulnerable Outdated Components
7. Identification and Authentication Failures
8. Software and Data Integrity Failures
9. Security Logging and Monitoring Failures
10. Server-Side Request Forgery

An additional resource to help you understand these categories more in-depth is the OWASP Cheat Sheet series.

2.2 Common Weakness Enumeration

The Common Weakness Enumeration (CWE) is a community-developed dictionary maintained by the MITRE Corporation. The security community has found it necessary to create a shared terminology amongst academics and professionals to better communicate weaknesses and vulnerabilities that can occur in system architecture, design, code, or implementation. It is sometimes referred to as a "measuring stick" for security tools, as the CWE acts as a common standard for identifying, mitigating, and preventing system weaknesses. [1]

The other large project run by MITRE is the Common Vulnerabilities and Exposures (CVE) program. CVEs are specific instances of a CWE (potential vulnerability) found within a system. One can think of a CWE as a class of a specific weakness, and the CVE is an instance of that CWE mapped to a specific software and version. Furthermore, OWASP has expanded on these efforts by identifying and categorizing CWEs into larger groups, and with the community's input, has created the *OWASP Top 10*, a list of the ten most common web application security risks. For example, under *A03: Injection*, all the CWEs associated with injections are listed.

These three tools, CWEs, CVEs, and OWASP Top 10, allow the researchers and practitioners to communicate clearly and effectively regarding security risks.

[1] For the most up-to-date list, please reference the CWE webpage at `https://cwe.mitre.org`.

2.3 CWE Mapping

During the course of your project, your team should conduct mapping to CWEs in all three phases.[2]

Phase I: Review the CWEs and intentionally design your system to defend against the listed weaknesses. For example, CWE-200 is *exposure of sensitive information to unauthorized parties*. Understanding this weakness, you might implement security controls that only allow privileged users to view and modify personally identifiable information. All team members should understand these mappings.

Phase II While implementing backdoors into your peer's system, look for weaknesses and identify matching CWEs to guide your attack strategy. For example, if you receive a voting machine that allows a non-privileged voter to access the registered voter database, identify this as CWE-200 and pursue this as a potential direction for vulnerabilities.

Phase III While making your security assessment to build your final presentation, use CWE mappings to assist in identifying and researching the vulnerabilities you discover. If you find that your peer's system has incorrectly allocated memory for a particular operation and is vulnerable to a buffer overflow attack, identify this as CWE-20, and use the respective documentation to guide your discussion and analysis.

3 Project Phases

3.1 Phase I: The Engineer

Working collaboratively, your team will design and build a system that is both functional and secure. Your instructor will assign you to a team (this will be your group for the entirety of the project) and randomly pair you with a system to build. The "system" is not a physical device; we expect your group to create a software package that provides the required functionality at a minimum. It could take the form of a command-line application, a web-based interface, or another format, depending on your group's decision and instructor approval – exercise creativity here! The system types are as follows:

- Automated Teller Machine (ATM)
- Slot Machine
- E-Voting System
- Smart Thermostat
- Automated Insulin Delivery (AID) System

All team members are expected to contribute equally to the project. Individual contributions will be reported to the instructor to ensure accountability.

3.1.1 Software Package I: The Implementation

The first package of code you write for this project will be a base implementation of the system you were assigned above. Additionally, your system must be secure to the OWASP Top 10 list in Section 2.1. As you develop your system ensure to regularly reference the list and build intentional defenses to each threat following secure-by-design principles. Remember: in the next phase of the project, another team will be attempting to *subvert* your system, so try to make their job as hard as possible!

This phase is your opportunity to be imaginative. To help you get started, reference the resources GitHub repository, which contains archived websites explaining the functionality of the real-world equivalent device and manuals for on-the-market devices. Use these materials to understand how your assigned system works, identify strengths and weaknesses, and design additional features!

[2] Ensure you and your team only use CWEs that are marked as "Vulnerability Mapping: Allowed" in the CWE list.

Requirements. Your requirements for the implementation code are as follows.

1. Your system must meet all the functional standards outlined in Section 5.
2. In addition to the functionality listed in Section 5, you are required to add at least one feature beyond these requirements, reflecting the real-world use of the device. The manuals in the the project resources GitHub repository can help provide inspiration.
3. Your system must include security controls for each OWASP Top 10 category (Section 2.1) relevant to its implementation. The functional requirements in Section 5 are designed to be expressive enough to encompass the Top 10; if you believe a Top 10 category is irrelevant, however, make sure to have a clear rationale as to why, as you will be asked to defend this decision.
4. Your system must be primarily written in *Golang* You may, however, include other programming languages (e.g., C) as sub-components. Also, you can use whatever Go libraries you would like to aid your implementation.
5. You must include installation instructions for your system. The installation instructions can assume Go is installed, but must guide the user through the installation of any relevant dependencies required (i.e., via `go mod tidy`). These are the instructions that must be followed exactly during the implementation demo day (Section 3.1.2), so make sure these are accurate.
6. You must include a user manual for your system. A well-written manual should guide users through all of the system's functions independently, without further assistance from the developers. You may combine the installation instructions and user manual into one document.

3.1.2 Implementation Demo Day

Your source code will be reviewed by the course instructor to verify functionality during the *implementation demo day* (Wednesday, November 18). Each group will have a 30-minute meeting with the instructor during class time. The first part will be a live demonstration of your system to the instructor showcasing its functionality. The second part will be an oral examination, during which your team should be prepared to answer questions about your system, justify design choices, explain each member's contributions, and discuss your system's security properties. Questions will be posed to team members at the discretion of the instructor, making the entire team's preparation critical for success. Your system must pass the implementation demo day to be advance to the next phase of the project.

Requirements. In the first part of the demo day, you will demonstrate that your Software Package I meets the functional requirements. Your demo must include the following elements:

1. Install the system according to your installation instructions. Any deviations from these instructions will be noted by the instructor.
2. Demonstrate how each functional requirement (as listed in Section 5 for your system) is met by your implementation. The instructor will use the list of functionality as a checklist to ensure each item is included in your implementation. It is up to your group to plan how to demonstrate all of the functionality, but you need to make sure that the instructor is convinced that each requirement is met.
3. Demonstrate that you have added an additional feature over what is required. You should be prepared to explain why this additional feature makes sense in the real-world context of your system.

All of the above steps must be completed within *10 minutes*, with each group member talking for about the same amount of time. If you run out of time, then you may not get full credit for your demo, so make sure to practice your timing. The remaining time for the demo day will be allocated for the oral examination. You and your group members must be prepared to do the following:

1. Defend your design choices, explaining how they meet the functional requirements of each system.
2. Describe the threats to your implementation posed by each category in the Top 10 list.
3. Explain how your implementation secures against the threats you identify above. You may be expected to point to specific lines of code.
4. If you believe that a Top 10 category does not apply to your system, justify why.
5. Given an arbitrary snippet of code from your implementation, describe what the code is doing and what the purpose of that code is in your implementation.
6. Explain your group collaboration dynamic: who did what, what elements of group collaboration were successful, and what elements you wish to improve upon for the next project phase.

Note that the instructor expects that *all* group members are expected to answer *every* question. Make sure you prepare extensively for the demo day!

3.1.3 Phase I Deliverables and Key Dates

- *Check-in 1* (due Monday, October 19). Should detail a general approach for your system implementation and handle group logistics. Brainstorm your approach to satisfy all functionality and security standards requirements. Establish and record a team-wide communication method.
- *Check-in 2* (due Friday, October 30). A preliminary skeleton of your system. Can be a visual control flow diagram, or a bulleted list. It should demonstrate consideration for all of the requirements and functionalities outlined for your respective machines, and provide a general framework for how it will be implemented. May be submitted as a clear photo of a sketched diagram or a document.
- *Implementation Meetings* (live and in-class)
- *Software Package I* (due Tuesday, November 17). A zipped file containing your system's source code, installation instructions, and a detailed user manual. To avoid including junk files in your submission, make sure to run `go clean` in your directory before zipping it for submission.
- *Implementation Demo Day* (live and in-class Wednesday, November 18). Your team will give a live, in-person demonstration of your system to your course instructor. Ensure that the system is entirely functional. All team members are expected to be prepared to answer questions regarding your design choices, CWE and OWASP mapping, and design and structure. It is highly recommended to meet with your team prior to ensure that all group members are equally proficient in both operating and explaining your system.

3.2 Phase II: The Adversary

Your team will receive another group's *Software Package I*. You will use adversarial thinking to reason through their system, identify potential modes of failure, and add vulnerabilities into the system that exploit the discovered weaknesses. To begin, once you receive the *Software Package I*, you need to understand the goal of the system and how the software components work to meet this objective.

Once you and your team have developed this understanding, you need to think like an adversary looking to subvert the system: "What would a threat actor want to achieve by targeting this system?" For example, imagine a keyless electric car, one of the things I would attempt to subvert would be the authentication procedure, such that I could unlock any car, no matter if I *should* have access or not. Following this process, your added vulnerabilities should then be constructed to meet your adversary's objective.

3.2.1 Software Package II: The Backdoors

The second package of code you write for this project will involve injecting backdoors into the original, secure system. This phase will require you to shift your thinking from defensive to offensive with subversion in mind.

Requirements. Your requirements for the backdoor code are as follows.

1. The system must remain functional and meet the same standards of functionality as in Phase I.
2. An updated user manual for the system that reflects any modifications to the system (e.g., user interface, account registration, etc.)
3. Your modified system should have the following vulnerabilities
    - A vulnerability from each of the following vulnerability classes below:
        - A01 Broken Access Control
        - A02 Cryptographic Failures
        - A03 Injection
    - Two from the four vulnerabilities listed below:
        - A05 Security Misconfiguration
        - A06 Vulnerable and Outdated Components
        - A08 Software and Data Integrity Failures
        - A09 Security Logging and Monitoring Failures
    - The following categories can be included in the subverted system; however, these will not be graded for points.
        - A04 Insecure Design
        - A07 Identification and Authentication Failures
        - A10 Server Side Request Forgery (SSRF)

3.2.2 Backdoor Demo Day

Backdoor demo day, Monday, December 7, is your team's second and final oral examination, following the same format as Demo Day One, but focused on demonstrating how you have subverted your assigned system to the course instructor. The examination consists of two distinct parts. First, you will deliver a presentation of your understanding of the system from an adversarial mindset and how your exploits align with this thinking. Additionally, you are expected to conduct a live demonstration of each vulnerability you've implemented.

Second, you'll participate in an oral examination where your team must be prepared to answer detailed questions about the system, your exploits, including OWASP categorization and CWE classifications, justify your subterfuge choices, and explain each member's specific contributions to the project. Questions will be directed to team members at the instructor's discretion, making comprehensive team-wide preparation essential for success. Passing the backdoor demo day is mandatory to advance to the next phase of the project, emphasizing the critical importance of thorough preparation and teamwork.

Requirements. In the first portion of the backdoor demo day, you will present your understanding of the system and the backdoors implemented in this phase. Your presentation must include the following elements:

1. Present your understanding of the system by discussing its functionality and what an adversary would like to accomplish by adding a backdoor.
2. Describe each vulnerability with respect to the system, an adversary's goal, OWASP categorization, and CWE classification.
3. Demonstrate that each vulnerability can be exploited in real-time.

All of the above steps must be completed within *10 minutes*, with each group member talking for about the same amount of time. If you run out of time, then you may not get full credit for your demo, so make sure to practice your timing. The remaining time for the demo day will be allocated for the oral examination. You and your group members must be prepared to do the following:

1. Defend your adversarial model and relation to the system, explaining how the goals of the adversary are applicable to the system
2. Explain how your vulnerabilities map to OWASP Top 10 categories and why the CWE classification is correct.
3. Explain your group collaboration dynamic: who did what, what elements of group collaboration were successful, how your group improved from the previous phase, and what elements you wish to improve upon in the last project phase.

3.2.3 Phase II Deliverables and Key Dates

- *Check-in 3* (due Tuesday, November 17) is to provide prompting questions to help your team get started analyzing a new system.
- *Software Package II* (due Sunday, December 6) A zipped file containing your system's modified source code and a detailed user manual.
- *Backdoor Demo Day* (due Monday, December 7) Your team will give a live, in-person presentation about your understanding of the system and its adversary. Additionally, you will review the backdoors you injected into the system with demonstrations of the exploits. As with the first demo day, it is recommended to meet with your team prior to ensure that all group members are equally proficient in both explaining the system and demonstrating the exploits.

3.3 Phase III: The Analyst

For this part of the project, you will be given one distribution of source code and a user manual from another group's *Software Package II*. Your team's goal is to perform an in-depth security analysis of the code to discover *all* vulnerabilities. Building upon the foundations established in the previous phases, you must develop an understanding of the system and, most importantly, an understanding of the adversary's subversive objectives.

One approach to solving this problem is to break down the system's exploration into stages. First, identify the system's components and their data flows. Then, for each component and data flow, identify the attack surface or threat. Taking a detailed pass through each component and data flow should help narrow the locations of the vulnerabilities. In addition to identifying the vulnerabilities, you must also generate the exploits for each vulnerability found.

3.3.1 Final Presentation

Your team's final presentation, Wednesday, December 23, is the culmination of your efforts. Your team must present your findings about the subverted system, including the vulnerabilities found, their exploits, and the OWASP categorization and CWE classifications. A detailed approach to the analysis is expected, with each member's specific contributions to the project mentioned. Clarifying

questions will be directed to team members at the instructor's discretion, making comprehensive team-wide preparation essential for success.

Requirements. During your final presentation, you will present your understanding of the system, the proposed adversary, the vulnerabilities, and their exploits. All of the above steps must be completed within *20 minutes*, with each group member talking for about the same amount of time. Make sure that you are able to talk for the full 20 minutes without going over time. Your presentation must include the following elements:

- A brief overview of the system and its functionality
- An analysis of the proposed adversary and their goals
- A description of your approach to finding vulnerabilities (e.g., components with user input, important data flows, pentesting tools, etc.)
- For each vulnerability, discuss the following:
  - A description of the vulnerability, including importance to the system, the type (from OWASP Top 10), and the CWE mapping
  - How you discovered this specific vulnerability
  - An explanation of the exploit and a proof-of-concept of the exploit, including inputs and screenshots as proof of exploitation.
  - How to mitigate the vulnerability (e.g., prepared statements, input validation and sanitization, etc.)

3.3.2 Phase III Deliverables and Key Dates

- *Check-in 4* (due Sunday, November 22) is to provide prompting questions to help your team get started analyzing a new system.
- *Check-in 5* (due Wednesday, December 9) is to help guide the remainder of the project in preparation for your final presentation.
- *Final Presentation* (due Wednesday, December 23) Your team will give a final presentation to the class about your security analysis of the assigned system. Your team should be able to articulate how you viewed the system from an adversarial perspective and how that guided your approach to analyzing the system. Additionally, your team should have a strong understanding of how to exploit each backdoor.

4 Check-Ins

To complete a check-in, discuss each question with your group. Then, distill your discussion into 2-3 sentences, and write up a response. Submit your responses to all questions to as a PDF. You may be asked during the demo days to defend your responses, so make sure to put some thought into them.

4.1 Check-in 1: Getting started

Due: Monday, October 19

The questions below are designed to get you thinking about how your group is going to design and implement your code.

1. In regular operation, what are the goals of the system you were assigned?

2. What are the errors and edge cases that you need to handle?
3. How will you test your code?

In addition to the implementation, your group should also consider project management, as the course project is a large-scale undertaking with several moving pieces. The below questions will establish a "group contract": a list of expectations to which the members of the group will pledge to adhere.

4. What is the contact information for your group members?
5. How will you break down the implementation into smaller, more manageable tasks?
6. During group selection, with which role did each group member most identify (engineer, adversary, analyst)?[3]
7. How will you ensure that each group member will contribute equally to the project?
8. How often will you meet? At what time will these standing meetings be? Where will you hold meetings (on campus, Zoom, etc.)?
9. How will you track progress on individual tasks?
10. What is your team name? This should be a collective group decision, and have fun with it!

Compile your responses to the above questions and submit it by the assigned deadline of Monday, October 19.

4.2 Check-in 2: Implementation design

Due: Friday, October 30

This first set of questions pertains to the design of your implementation.

1. What is the architecture of your solution? Describe each part of your implementation, and how they connect to each other to provide the required functionality. You may include a diagram of your solution to aid your description.
2. How is your code organized? What modules have been defined for each functionality?
3. What is the user interface for the project? Will it be a command-line interface, a terminal application, a graphical application, a website, or something else?
4. What additional feature(s) do you plan on adding to your implementation?
5. What is the relevance of these additional feature(s)? Cite outside sources in your response.

The next set of questions is related to your group's dynamics and project management.

6. Which members of your group have been assigned to each task?
7. What tasks are remaining?
8. When do you expect each task to be completed?

This final set of questions are designed to help you prepare for the demo session. They are similar in theme to the questions that will be asked of you during your oral examination.

[3] Remember, while some group members may identify more closely with a role than others, everyone is expected to partake in *all* roles throughout the course of the project and to participate equally in terms of contribution. But, those who identify with a role will be considered a lead on that phase, and should coordinate the other group members' tasks.

9. Go through the functionality requirements in Section 5 for your assigned application. What features does your code support? What features still need to be implemented?
10. What happens when your code encounters an incorrect input? How have you tested this?
11. For each category of the OWASP Top 10, describe how your code is designed to defend against attacks from the category.[4]
12. Are you happy with your code's organization? Why or why not?
13. Are your installation instructions detailed enough to configure your code? Explain.
14. On the demo day, each group member will be asked to describe their contributions to the codebase. What will each group member discuss as their contribution? Note that it is expected that each group member contributes equally to the codebase.

Compile your responses to the above questions and submit it by the assigned deadline of Friday, October 30.

4.3 Check-in 3: Backdooring a system

Due: Tuesday, November 17

You have been assigned another group's system and codebase. This set of questions is designed to help you start thinking about how you will adversarially subvert the system by adding vulnerabilities.

1. In regular operation, what are the goals of the system you were given?
2. How do the software components interact to achieve these goals?
3. Thinking adversarially, why would a threat actor want to target this system to subvert it?
4. What specific objective(s) would a threat actor want to achieve by targeting this system?
5. What parts of the codebase could you change to subvert the system and achieve the threat actor's objective(s)?
6. Based on the OWASP Top 10 (Section 2.1) and the requirements for this phase (Section 3.2.1), what kinds of vulnerabilities will you add? Mention specific CWEs in your response.
7. Why would these vulnerabilities be appropriate? Note that the vulnerabilities need to "subvert the system" (Section 3.2), not just crash the program.
8. The project description also mentions that the vulnerabilities need to be hidden from a regular user. How could you hide the vulnerability in your user interface without alerting a user to its presence?
9. What are some ways you can make your vulnerabilities will be hard to identify and exploit by the red team? You may have to do some outside research for this.

Compile your responses to the above questions and submit it by the assigned deadline of Tuesday, November 17.

[4] Note that not all Top 10 categories will apply to all systems. For instance, Server-Side Request Forgery may not apply to designs without multiple server backends. Before you conclude that a Top 10 category does not apply to your system, discuss as a group and come to an agreement as to the reason why, and then include this rationale in your check-in response.

4.4 Check-in 4: Analyzing a system

Due: Sunday, November 22

You have been assigned yet another group's system and codebase. Now, we will analyze it by going through each category of the OWASP Top 10, looking for vulnerabilities of that type. This check-in is designed to help you with that process.

1. In regular operation, what are the goals of the system you were given?
2. What parts of the code correspond to the required functionality?
3. What parts of the code appear not to be related to the required functionality? This is likely where the vulnerabilities can be found.
4. Can you use the user interface to run the code corresponding to the different functionality? This could mean that any vulnerabilities can be exploited.
5. What vulnerabilities have you found in the code so far? How are they exploited?
6. Map each vulnerability you have found to a category of the OWASP Top 10.

Compile your responses to the above questions and submit it by the assigned deadline of Sunday, November 22.

4.5 Check-in 5: Presenting your results

Due: Wednesday, December 9

This final check-in is designed to help you plan the remainder of the project.

1. Map each vulnerability you have found to a specific CWE.[5] Explain why your chosen CWE is appropriate for each vulnerability found. Make sure you only map to CWEs that are marked as "Vulnerability Mapping: Allowed".
2. What is your group's timeline to finish the red team analysis and create the presentation? When and where will you meet?
3. What will be the content of your presentation? Go through Section 3.3 before answering this question, so that your group is aware of the requirements.
4. Who will present which part of the presentation? Note that every group member must present for about an equal amount of time.

Compile your responses to the above questions and submit it by the assigned deadline of Wednesday, December 9.

5 System Specifications

5.1 Automated Teller Machine (ATM)

You are **[choose a team name!]**, an up-and-coming financial software company. You have been contracted by Big Bank to create a secure ATM machine. Your task is to build an ATM that meets their requirements, while innovating and expanding to create a high-quality, unique machine that can be deployed at their hundreds of branch locations. Big Bank's functionality requirements are as follows:

[5] Each Top 10 item has a list of associated CWEs, which can be found in the linked files in Section 2.1.

- The ATM should not be a physical machine, but an executable *Golang* software package. A CLI interface is suggested, but feel free to write a web front-end if desired.
- The system should primarily be written in *Golang* , but sub-components written in other languages are permissible.

  For example, if you need a database, consider SQLite for easy integration.
- You may use any available *Golang* libraries in your implementation.
- The ATM must have 3 user groups: **Teller Admins**, **Customers and Cash Handler.**

Teller Admins must be able to:

- Authenticate into an admin-only interface.
- Create a new customer account with the following required information:
  - Full name
  - Date of birth
  - Numerical PIN of their choosing
  - Starting balance
- View logs of all deposits and made to the bank.
- Set daily withdrawal and deposit limits for all customers.

Customers must be able to:

- Log into their ATM account using their PIN number.
- View the total balance of their account.
- View the withdrawal and transfer limit.
- Withdraw funds from their account.
- Select their desired withdraw denomination.
  - Customers cannot withdraw more than the total balance of their account or the withdrawal limit.
- Deposit funds into their account.
  - Customers should not be able to deposit more than the set daily limit.
- Transfer funds to other customers within the bank.
  - Customers must have sufficient funds in their account to fulfill the transfer.
  - Target transfer account should be validated.

Cash Handlers must be able to:

- Authenticate into a secure cash-handler only interface.
- View the cumulative cash total for the machine, itemized by bill denomination.
- Deposit large amounts of cash to restock the machine. Make sure to take into account the customer daily withdraw maximum, and the cumulative ATM cash total.
- Withdraw large amounts of cash to reset the machine to the cash maximum.

Tips for success:

- When brainstorming your system, try drawing a control flow diagram for both admins and customers. Visualizing the path of decisions can help you think through your design. Review this guide for diagramming code and drawing control flows to get started!
- When working in groups, it is highly advisable to thoroughly comment your code throughout to prevent confusion.
- Make sure to test your system often!
- Review your mobile banking app for inspiration. What questions are you asked to make a mobile deposit? Think about how this could translate into your ATM.

5.2 Slot Machine

You are **[choose a team name!]**, an up-and-coming money printing company. You have been contracted by Big Casino to design and build a secure and fair slot machine. Your task is to create a system that meets Big Casino's design specification requirements while being innovative and providing an easy-to-use user experience. Big Casino requires the following:

- Your slot machine should not be a physical machine, but an executable *Golang* software package. A CLI interface is suggested, but feel free to write a web front-end if desired.
- The system should primarily be written in *Golang* , but sub-components written in other languages are permissible.

  For example, if you need a database, consider SQLite for easy integration.
- You may use any available *Golang* libraries in your implementation.
- The slot machine must support three user groups: **Big Casino Technicians**, **Big Gamble Commissioners** and **Players**.

Big Casino Technicians must be able to:

- Authenticate into a technician-only interface.
- Manage games (add or remove) from the playable interface
  - There should be a minimum of **two** different slot games to be played.
  - Each game should have multiple betting options ($1, $5, etc.).
  - At least one game should support multiple win conditions, such as diagonals, multiple rows, etc.
  - At least one game to allow for holds on wheels for the next spin.
  - All random number generation needs to be cryptographically secure.
- Implement system logging for auditing purposes.
- Run a system diagnostic to display the wins and losses for each game, the frequency of game play, and other relevant information.

Big Gamble Commissioners must be able to do:

- Authenticate into a testing-only mode
- Analyze the statistics of each game type, the wins and losses, frequency of payout, and other relevant information.

- Test a game for x rounds, playing the specific game x times, and calculate the statistics for the test session, including the random seeds used for each round.

Players must be able to do:

- Register a new account and manage the account
  - Connect the account to a bank account or debit/credit card
  - Deposit funds or withdraw funds to and from the bank account
  - Manage the password for the account
- Play one of the slot machine games
  - Bet money if their account has sufficient funds
  - Cash out of the game for funds to be added to their account
- View their playing statistics such as number of games played, highest payout, total lost, win percentage, etc.

Tips for success:

- Review the Pokémon Game Corner slots for ideas.
- Review Secure Random Number Generation for information about secure random generation.

5.3 E-Voting System

You are **[choose a team name!]**, an up-and-coming software company. You have been contracted by Big Vote to design and build a secure voting machine. Your task is to create a system that not only meets Big Vote's requirements but innovates and expands on them to deliver a high-quality machine to secure a long-term contract. Big Vote's functionality requirements are as follows:

- Your voting machine should not be a physical machine, but an executable *Golang* software package. A CLI interface is suggested, but feel free to write a web front-end if desired.
- The system should primarily be written in *Golang* , but sub-components written in other languages are permissible.

  For example, if you need a database, consider SQLite for easy integration.
- You may use any available *Golang* libraries in your implementation.
- The voting machine must have 3 user groups: **Election Admins, District Officials, and Voters**.

Election Admins must be able to:

- Authenticate into an admin-only interface.
- Create a new election.
  - Each new election should support at least 3 customizable positions for election (e.g., President, Vice President, Secretary or Best Sandwich, Best Taco, Best Hamburger).
  - Each position should have at least 2 possible candidates for election (e.g., Subway, Jersey Mike's, Firehouse Subs).
  - Each candidate must have fields for a name and political party.
- Register new voters.
  - Create a new voter using a full name and date of birth.

- Check that this voter does not already exist.

Voters must be able to:

- Only cast votes in an open election.
- Cast 1 vote per candidacy position.
- Once cast, votes should be immutable.

District Officials must be able to:

- Open an existing election to accept votes.
- Close an ongoing election to end ballot collection.
- Tally the results and calculate the winner for each position.
- Display winners for all candidacy positions.

Tips for success:

- When brainstorming your system, try drawing a control flow diagram for both admins and users. Visualizing the path of decisions can help you think through your design. Review this guide for diagramming code and drawing control flows to get started!
- When working in groups, it is highly advisable to thoroughly comment your code throughout to prevent confusion.
- Test your system often!
- The U.S Election Assistance Commission publishes detailed guides for secure, accessible voting systems. Skim one for inspiration!

5.4 Smart Thermostat

You are **[choose a team name!]**, an up-and-coming Internet of Things (IOT) device company. You have been contracted by Big Home to design and build a secure smart thermostat. Your task is to create a system that meets Big Home's design specification requirements, but is innovative and provides an easy-to-use user experience. Big Home requires the following:

- Your smart thermostat should not be a physical machine, but an executable *Golang* software package. A CLI interface is suggested, but feel free to write a web front-end if desired.
- The system should primarily be written in *Golang* , but sub-components written in other languages are permissible.

  For example, if you need a database, consider SQLite for easy integration.
- You may use any available *Golang* libraries in your implementation.
- The smart thermostat must support three user groups: **Homeowners**, **Guests**, and **Big Home technicians**.

Homeowners must be able to:

- Authenticate into a homeowner-only interface with secure credentials
- Interface with the smart thermostat
  - Indoor temperature sensing and other environmental monitoring (e.g., humidity, CO levels, etc.)

  - Retrieve outdoor weather statistics (e.g., temperature, humidity, precipitation, etc.)
  - HVAC system control (heating, cooling, fan)
  - Scheduling and programmable temperature control
  - The thermostat should be able to be programmed against profiles. For example, the homeowner can have a "vacation" profile that responds
  - Energy usage tracking and reporting
- Create and manage guest accounts - name, password/PIN
- Create and manage smart thermostat profiles
- Allow a specific technician(s) to access the system for a specified time period

Technicians must be able to:

- Authenticate into the technician-only interface with secure credentials with permission from the homeowner
- Interface with the smart thermostat as noted above
- Run system diagnostics for functionality
- Create and manage guest accounts - name, password/PIN
- Create and manage smart thermostat profiles

Guests must only be able to:

- Authenticate a guest using a PIN
- Interface with the smart thermostat with limited capabilities
  - Indoor temperature sensing and other environmental monitoring (e.g., humidity, CO levels, etc.)
  - Retrieve outdoor weather statistics (e.g., temperature, humidity, precipitation, etc.)
  - HVAC system control (heating, cooling, fan)
- Select smart thermostat profile

Tips for success

- Consider the privacy leakage that can exist when a guest has more access than necessary
- Review the Home Assistant for functionality and home automation ideas.

5.5 Automated Insulin Delivery (AID) System

You are **[choose a team name!]**, an up-and-coming medical device manufacturer. You're going to revolutionize diabetes care by creating an open-source insulin pump firmware that the community will validate. Your task is to develop the insulin pump firmware that meets regulatory and patient safety requirements, while providing the diabetes community with a reference implementation for continuous glucose measurement (CGM) analysis, high and low blood sugar alarms, and insulin pumping.

Specific requirements for the AID System are as follows:

- The insulin pump should not be a physical device, but an executable *Golang* software package that simulates CGM reading, alarms, and pumping behavior.

- CGM readings represent real-time blood glucose levels or blood sugar in the blood; readings fall within the following ranges:

  Normal Range: 70-180 mg/dL (milligrams per deciliter)

  High Glucose (Hyperglycemia): Above 180 mg/dL

  Low Glucose (Hypoglycemia): Below 70 mg/dL

- Pumping behavior encompasses bolus (i.e., a quick-acting dose administered with meals) and basal (i.e., a long-acting daily dose) injections.

- A CLI interface is required to exercise CGM value reading and pumping based on user-supplied data via STDIN, arguments, socket, or file.

  For example, a file may describe bulk readings for an entire day, where each entry is on a new line and represents a reading taken every minute. A user (you) or a program may use STDIN or a socket to provide readings at arbitrary intervals.

- A web or graphical front-end is optional; it should provide configuration and management of the AID System, similar to the CLI, and offer data charting or other outputs.

  To be clear, the front-end shall have the same functionality as the CLI with additional UI/UX elements that include blood glucose charting, visual and audible alerts, etc. The intended use of a UI for the AID System is for the user groups defined below to manage diabetes care safely.

- The system should primarily be written in *Golang* , but sub-components written in other languages are permissible.

  For example, if you need a database to store glucose readings or logs, consider SQLite.

  You may also need a "test harness" for feeding reproducible data into your system. You may implement this harness in a shell or Python script.

  You may use any available *Golang* libraries in your implementation.

- All AID System operations must be logged, especially those with patient safety concerns, including but not limited to: insulin basal and bolus injections, insulin configuration changes, and high and low glucose alerts

  You must consider what information to log with these events. For example, the user who executed the function and the timestamp of when they did it are critical for later analysis.

- The insulin pump must have three user groups: **Clinician**, **Caretaker**, and **Patient**.

Clinicians must be able to:

- Authenticate using a username and password in the CLI or web front-end.
  - Assume multiple clinicians per AID System.
- Create a new patient account with the following required information:
  - Full name
  - Date of birth
  - Unique patient ID
  - Initial basal insulin rate (i.e., daily insulin which may be 5 units per meal)
  - Initial bolus insulin rate (i.e., base insulin per meal which may be 30 units per day)
- View logs of all insulin deliveries and glucose readings.
- Set dosage limits, safety thresholds, and alert conditions.
- Adjust insulin delivery parameters based on patient needs.

- Set alert defaults for low and high blood sugar events.

Caretakers must be able to:

- Log into the user's AID System account using a unique PIN or password.
  - Assume one caretaker per AID System.
  - Assume a simplified authentication system for the target population which may include older.
- View the patient's most recent glucose readings.
- View the patient's current basal rate and bolus insulin options.
- Request a bolus insulin dose.
  - Caretakers cannot request more than the prescribed maximum dose or violate safety limits.
  - Caretakers cannot request more than one dose per every four hours (corresponding to three meals a day).
- Configure basal insulin dose time.
  - Caretakers can adjust the basal insulin dose, which will be effective within 24 hours, so as not to overlap a previous dose.
  - Caretakers cannot request more than the prescribed maximum dose or violate safety limits.
- Review historical insulin delivery and glucose data.
- Receive alerts when glucose levels fall outside safe thresholds.

Patients must be able to:

- Log into their AID System account using a PIN or password.
  - Assume one patient per AID System.
  - Assume a simplified authentication system for the target population which may include older or disabled users.
- View their most recent glucose readings.
- View their current basal rate and bolus insulin options.
- Request a bolus insulin dose.
  - Patients cannot request more than the prescribed maximum dose or violate safety limits.
- Configure basal insulin dose time.
  - Patients can adjust the basal insulin dose, which will be effective within 24 hours, so as not to overlap a previous dose.
  - Patients cannot request more than the prescribed maximum dose or violate safety limits.
- Review historical insulin delivery and glucose data.
- Receive alerts when glucose levels fall outside safe thresholds.

Tips for success:

- When brainstorming your system, try drawing a control flow diagram for clinicians, caretakers, and patients. Visualizing the path of decisions can help you think through your design. Review this guide for diagramming code and drawing control flows to get started!

- Consider how cybersecurity applies to medical safety—authentication, integrity, secure data logging, and preventing unauthorized commands are essential.
  - What happens if the CGM data that informs the insulin dosage is maliciously modified or incorrect?
  - What could a malicious caretaker do to affect the patients safety?
  - How would an attacker attempt to exploit vulnerabilities that compromise the patient's safety and hide it?
- When working in groups, it is highly advisable to comment your code throughout to increase readability and reduce code quality issues, such as dead code and redundancy.
- Make sure to test your system regularly, especially around edge cases such as maximum dose requests and abnormal glucose values.
- Review FDA cybersecurity guidance for secure product development for inspiration. Think about what protections should exist to prevent malicious misuse of a life-critical device.